StrengthLabs

Documentacion Tecnica Completa

Backend · Motor de Computo · Aplicacion Movil

Version 1.0

8 de abril de 2026

Repositorio backend:	`StrengthLabsBackend/`	
Repositorio frontend:	`strengthlabs_beta/`	
Puerto API:	`8000`	Uso interno. Todos los derechos
Puerto computo:	`8001`	
Base de datos:	PostgreSQL 16 en `5432`	

reservados.

Índice

1. Vision general del sistema **4**
1.1. Stack tecnologico . . . 4
1.2. Servicios y puertos . . . 4
1.3. Flujo general de datos . . . 4

2. Backend: StrengthLabsBackend (Java/Spring Boot) **5**
2.1. Arquitectura limpia . . . 5
2.2. Seguridad y autenticacion . . . 5
2.2.1. JWT . . . 5
2.2.2. CORS y sesiones . . . 5
2.2.3. Rutas publicas . . . 6
2.3. Entidades de base de datos . . . 6
2.3.1. users . . . 6
2.3.2. exercises . . . 6
2.3.3. workouts . . . 6
2.3.4. workout_exercises . . . 7
2.3.5. workout_sets . . . 7
2.4. Endpoints REST . . . 7
2.4.1. Autenticacion — `/auth` . . . 7
2.4.2. Sesiones de entreno — `/workouts` . . . 7
2.4.3. Ejercicios — `/exercises` . . . 8
2.4.4. Fatiga — `/fatigue` . . . 8
2.4.5. Rutinas — `/routines` . . . 9
2.4.6. Exportacion — `/export` . . . 10
2.4.7. Administracion — `/api/v1/admin` . . . 10
2.5. Catálogo de ejercicios (seeder) . . . 11
2.6. Rutinas predefinidas . . . 11
2.7. Configuracion . . . 11
2.7.1. application.yml (por defecto) . . . 11
2.7.2. application-dev.yml . . . 12
2.7.3. Variables de entorno requeridas . . . 12

3. Motor de computo: Python/FastAPI (puerto 8001) **12**
3.1. Endpoints . . . 12
3.2. Modelo de fatiga . . . 13
3.2.1. TRIMP y carga de sesion . . . 13
3.2.2. EWMA: ATL y CTL . . . 13
3.2.3. Monotonia y strain . . . 13
3.2.4. Readiness score . . . 13
3.2.5. Banderas de riesgo de fatiga . . . 14
3.3. Modelo de riesgo . . . 14
3.3.1. Puntuacion de riesgo de lesion . . . 14
3.3.2. Puntuacion de riesgo de sobreentrenamiento . . . 14
3.3.3. Puntuacion compuesta . . . 14
3.4. Modelo de periodizacion . . . 15
3.4.1. Fases de entrenamiento . . . 15

3.4.2. Multiplicadores de carga . . . 15
3.4.3. Salida del plan semanal . . . 15

4. Frontend: strengthlabs_beta (Flutter) 16
4.1. Estructura de directorios . . . 16
4.2. Infraestructura core . . . 17
4.2.1. DioClient . . . 17
4.2.2. TokenStorage . . . 18
4.2.3. ApiConstants . . . 18
4.2.4. AppRouter . . . 18
4.3. Entidades de dominio . . . 19
4.3.1. User . . . 19
4.3.2. Exercise . . . 19
4.3.3. WorkoutSet . . . 20
4.3.4. WorkoutExercise . . . 20
4.3.5. Workout . . . 20
4.3.6. FatigueSummary . . . 20
4.3.7. Routine . . . 21
4.3.8. TrainingPlan . . . 21
4.4. Gestion de estado (Cubits) . . . 22
4.4.1. AuthCubit . . . 22
4.4.2. WorkoutsCubit . . . 22
4.4.3. ActiveWorkoutCubit . . . 22
4.4.4. FatigueCubit . . . 23
4.4.5. RoutinesCubit . . . 23
4.4.6. PlanCubit . . . 23
4.5. Capa de datos (Repositorios) . . . 24
4.5.1. AuthRepository . . . 24
4.5.2. WorkoutRepository . . . 24
4.5.3. FatigueRepository . . . 24
4.5.4. RoutineRepository . . . 24
4.5.5. ComputeRepository (calculo offline) . . . 25
4.5.6. ExportService . . . 25
4.6. Pantallas . . . 25

5. Flujo de autenticacion 26

6. Configuracion del entorno de desarrollo 26
6.1. Requisitos previos . . . 26
6.2. Puesta en marcha . . . 27
6.2.1. 1. Infraestructura (PostgreSQL + Redis + Motor de computo) . . . 27
6.2.2. 2. Backend Java . . . 27
6.2.3. 3. Aplicacion Flutter . . . 27
6.3. Nota sobre URLs en emuladores . . . 27

7. Decisiones de diseno destacadas 27

A. Catalogo completo de ejercicios globales 28

B. Rutinas predefinidas: detalle de dias **30**
B.1. r-beg-strength: 3-Day Beginner Strength 30
B.2. r-int-hyp: 4-Day Upper/Lower Hypertrophy 31

1 Vision general del sistema

StrengthLabs es una plataforma para el registro, analisis y planificacion del entrenamiento de fuerza. Consta de tres componentes independientes que se comunican a traves de HTTP:

1. **StrengthLabsBackend** — API REST en Java 21 / Spring Boot 4, responsable de la autenticacion, persistencia y coordinacion.
2. **Motor de computo** — microservicio Python/FastAPI que ejecuta los algoritmos de carga de entrenamiento (ATL/CTL/ACWR), riesgo y periodizacion.
3. **strengthlabs_beta** — aplicacion movil Flutter (iOS y Android) que consume la API y presenta la informacion al atleta.

1.1 Stack tecnologico

Capa	Tecnologia
Backend Java	Java 21, Spring Boot 4.0, Spring Data JPA, Spring Security
ORM / BD	Hibernate 6, PostgreSQL 16
Cache	Redis 7
Seguridad	JWT HS256 (JJWT 0.12), BCrypt, CORS abierto
Documentacion API	SpringDoc OpenAPI 2.8 (Swagger UI en `/swagger-ui.html`)
Exportacion	Apache POI 5 (XLSX)
Motor de computo	Python 3.11, FastAPI, NumPy, SciPy
Contenedores	Docker Compose (postgres, redis, compute-engine)
Frontend	Flutter 3.x (iOS y Android)
Estado (frontend)	Flutter Bloc / Cubit 8.x
Navegacion	GoRouter 13.x (`StatefulShellRoute`, 5 tabs)
HTTP client	Dio 5.x con interceptor JWT
Tokens (frontend)	flutter_secure_storage 9 (Android encriptado)
Exportacion local	Paquetes `excel` + `share_plus`

1.2 Servicios y puertos

Servicio	Host	Puerto	Notas
API Backend	localhost	8000	Spring Boot
Motor de computo	localhost	8001	FastAPI (Docker)
PostgreSQL	localhost	5432	Docker, db `strengthlabs`
Redis	localhost	6379	Docker

1.3 Flujo general de datos

1. El atleta registra un entreno en la app → POST `/workouts` → persiste en PostgreSQL.

2. Al abrir el dashboard de fatiga → GET `/fatigue/summary` → el backend agrega los ultimos 90 dias y llama al motor de computo (`/compute/fatigue` + `/compute/risk`).
3. La pantalla Plan lee las metricas de la fatiga y genera localmente un plan semanal por bloques de periodizacion.
4. El usuario exporta su historial → GET `/export/csv` o `/export/xlsx` → el backend lee la BD y devuelve el archivo.

2 Backend: StrengthLabsBackend (Java/Spring Boot)

2.1 Arquitectura limpia

El backend sigue *Clean Architecture* con cuatro capas:

domain/ Entidades puras (**`User`**, **`WorkoutSession`**, **`TrainingPlan`**), repositorios como interfaces, servicios de dominio (**`PeriodizationService`**) y value objects (**`FatigueScore`**, **`RiskLevel`**).

application/ Casos de uso, DTOs de entrada (**`SessionInputDTO`**) y puertos (**`ComputeEnginePort`**).

infrastructure/ Adaptadores JPA, adaptador Python (**`PythonComputeAdapter`**), filtro JWT (**`RbacFilter`**), integracion Redis y seeder de ejercicios.

presentation/ Controladores REST con validacion Jakarta Bean Validation.

2.2 Seguridad y autenticacion

2.2.1 JWT

- **Algoritmo:** HMAC-SHA256 (HS256) via JJWT 0.12.
- **Claims:** `sub` (UUID del usuario), `role` (USER/TRAINER/ADMIN), `type` (access | refresh).
- **Expiracion access token:** 24 horas (configurable via `jwt.expiration-ms`).
- **Expiracion refresh token:** 7 dias.
- **Secreto:** variable de entorno `JWT_SECRET`; fallback de desarrollo `dev-secret-key-must-be-at-least-`

2.2.2 CORS y sesiones

- CORS: todos los origenes, todos los metodos (GET/POST/PUT/DELETE/OPTIONS), todos los headers; sin credenciales.
- CSRF: deshabilitado.
- Sesion HTTP: `STATELESS` (solo JWT).

- Contrasena: BCrypt con factor de coste por defecto.

2.2.3 Rutas publicas

`/auth/**`, `/swagger-ui/**`, `/v3/api-docs/**`. Todo lo demas requiere `Authorization: Bearer <token>`.

2.3 Entidades de base de datos

2.3.1 users

Campo	Tipo	Notas
`id`	UUID	PK, generado
`name`	VARCHAR	NOT NULL
`email`	VARCHAR(255)	UNIQUE NOT NULL
`password_hash`	TEXT	BCrypt
`role`	ENUM	USER / TRAINER / ADMIN
`active`	BOOLEAN	DEFAULT true

2.3.2 exercises

Campo	Tipo	Notas
`id`	UUID	PK
`name`	VARCHAR	NOT NULL
`muscle_group`	VARCHAR	chest, back, shoulders, etc.
`is_custom`	BOOLEAN	false = global, true = usuario
`created_by`	UUID	NULL para ejercicios globales

2.3.3 workouts

Campo	Tipo	Notas
`id`	UUID	PK
`user_id`	UUID	FK users
`name`	VARCHAR	NOT NULL
`date`	TIMESTAMPTZ	NOT NULL
`duration_seconds`	INTEGER	
`notes`	TEXT	nullable

2.3.4 workout_exercises

Campo	Tipo	Notas
`id`	UUID	PK
`workout_id`	UUID	FK workouts, cascade delete
`exercise_id`	UUID	FK exercises, eager load
`order_index`	INTEGER	orden de presentacion

2.3.5 workout_sets

Campo	Tipo	Notas
`id`	UUID	PK
`workout_exercise_id`	UUID	FK workout_exercises
`weight_kg`	DOUBLE	nullable (peso cero permitido)
`reps`	INTEGER	NOT NULL
`rpe`	DOUBLE	nullable, escala 6–10
`order_index`	INTEGER	orden dentro del ejercicio

2.4 Endpoints REST

Todos los endpoints protegidos requieren `Authorization: Bearer <access_token>`. Las respuestas de error siguen el formato de Spring (`status`, `error`, `message`).

2.4.1 Autenticacion — `/auth`

Metodo	Ruta	Cuerpo de solicitud	Respuesta
POST	`/auth/register`	`{name, email, password}`	`{access_token, refresh_token}`
POST	`/auth/login`	`{email, password}`	`{access_token, refresh_token}`
POST	`/auth/refresh`	`{refresh_token}`	`{access_token, refresh_token}`
GET	`/auth/me`	—	`{id, name, email}`

Errores notables: 401 credenciales invalidas, 409 email ya registrado.

2.4.2 Sesiones de entreno — `/workouts`

Metodo	Ruta	Descripcion
GET	/workouts	Lista de sesiones del usuario autenticado, orden descendente por fecha. Responde {items: [workout]}.
POST	/workouts	Crea nueva sesion. Cuerpo: {name, date (ISO8601), duration_seconds, notes, exercises[]}. Cada ejercicio: {exercise_id (UUID), sets[], order}. Cada set: {weight, reps, rpe, order}.
GET	/workouts/{id}	Detalle con ejercicios y series anidados.
PUT	/workouts/{id}	Actualiza name y/o notes.
DEL	/workouts/{id}	Elimina la sesion (204 sin cuerpo).

Estructura del objeto workout en respuestas:

```
{
  "id": "uuid",
  "name": "Push Day A",
  "date": "2025-04-08T10:00:00Z",
  "duration_seconds": 4500,
  "notes": "...",
  "exercises": [
    {
      "exercise": { "id": "uuid", "name": "Bench Press",
                    "muscle_group": "chest", "is_custom": false },
      "sets": [
        { "id": "uuid", "weight": 90.0, "reps": 5, "rpe": 8.5 }
      ]
    }
  ]
}
```

2.4.3 *Ejercicios — /exercises*

Metodo	Ruta	Descripcion
GET	/exercises	Devuelve lista unificada: ejercicios globales + personalizados del usuario. Cada elemento: {id, name, muscle_group, is_custom}.
POST	/exercises	Crea ejercicio personalizado. Cuerpo: {name, muscle_group}. Responde 201 con el ejercicio creado.

2.4.4 *Fatiga — /fatigue*

Metodo	Ruta	Descripcion
GET	/fatigue/summary	Agrega los ultimos 90 dias de entrenos, llama al motor de computo (/compute/fatigue y /compute/risk) y devuelve el objeto completo de metricas (ver Seccion 2.4.4).
GET	/fatigue/weekly	Volumen semanal (ultimos 7 dias) por grupo muscular, normalizado a 0–100 % de 20 series/-semana. Responde {weekly_volume: {chest: 45.0, ...}}.

El objeto de respuesta de /fatigue/summary:

```
{
  "overall_index":            float,   // readiness 0-100
  "is_overtraining":          bool,
  "weekly_volume":            { "chest": float, "back": float, ... },
  "trend":                    [{ "date": "YYYY-MM-DD", "index": float
      }],
  "atl":                      float,   // carga aguda (7d EWMA)
  "ctl":                      float,   // carga cronica (42d EWMA)
  "tsb":                      float,   // balance: CTL - ATL
  "acwr":                     float,   // ratio agudo/cronico
  "monotony":                 float,   // media/desv.est. ultimos 7d
  "strain":                   float,   // carga total x monotonia
  "ramp_rate":                float,   // delta CTL ultimos 7d
  "readiness_score":          float,   // 0-100 compuesto
  "risk_flags":               [string],
  "injury_risk_score":        float,   // 0.0-1.0
  "overtraining_risk_score": float,
  "composite_risk_score":     float,
  "risk_level":               "low" | "moderate" | "high" | "critical
      ",
  "dominant_factor":          string,
  "recommendations":          [string]
}
```

2.4.5 Rutinas — /routines

Metodo	Ruta	Descripcion
GET	/routines	Lista de rutinas predefinidas. Param opcional: ?level=beginner|intermediate|advanced. Responde {items: [routine]}.
GET	/routines/{id}	Detalle de una rutina con dias y ejercicios.

Las rutinas son datos estaticos embebidos en el controlador (no persisten en BD). Estructura del objeto rutina:

```
{
  "id": "r-beg-strength",
  "name": "3-Day Beginner Strength",
  "level": "beginner",
  "goal": "strength",
  "days_per_week": 3,
  "description": "...",
  "days": [
    {
      "name": "Day A - Full Body",
      "exercises": [
        {
          "exercise": { "id": "ex-...", "name": "Barbell Back Squat"
              ,
                        "muscle_group": "legs", "is_custom": false
                           },
          "sets": 3,
          "reps_scheme": "5",
          "notes": null
        }
      ]
    }
  ]
}
```

2.4.6 Exportacion — `/export`

Metodo	Ruta	Descripcion
GET	`/export/csv`	Exporta el historial completo del usuario como CSV. Nombre de archivo: `strengthlabs_export.csv`. Content-Type: `text/csv`.
GET	`/export/xlsx`	Exporta como hoja de calculo Excel. Nombre de archivo: `strengthlabs_export.xlsx`. Content-Type: `application/vnd.openxmlformats...`.

Columnas exportadas: *Date, Workout, Exercise, Muscle Group, Set #, Weight (kg), Reps, RPE, Volume (kg).*

2.4.7 Administracion — `/api/v1/admin`

Metodo	Ruta	Descripcion
DEL	`/api/v1/admin/users/{id}/deactivate`	Desactiva la cuenta de un usuario. Requiere rol ADMIN.

2.5 Catálogo de ejercicios (seeder)

El componente **`ExerciseDataSeeder`** inserta los ejercicios globales al arrancar si la tabla esta vacia (operacion idempotente). Se incluyen **62 ejercicios** en 11 grupos musculares:

Grupo	Ejercicios
Chest (7)	Barbell Bench Press, Incline Barbell Press, Incline Dumbbell Press, Dumbbell Bench Press, Cable Fly, Push-up, Dip
Back (10)	Barbell Deadlift, Barbell Row, Pendlay Row, Dumbbell Row, Lat Pulldown, Seated Cable Row, Pull-up, Weighted Pull-up, T-Bar Row, Romanian Deadlift
Shoulders (7)	Overhead Press, Dumbbell Shoulder Press, Arnold Press, Lateral Raise, Front Raise, Face Pull, Rear Delt Fly
Biceps (6)	Barbell Curl, Dumbbell Curl, EZ Bar Curl, Hammer Curl, Preacher Curl, Incline Dumbbell Curl
Triceps (5)	Tricep Dip, Tricep Pushdown, Skull Crusher, Close-Grip Bench Press, Overhead Tricep Extension
Legs (9)	Barbell Back Squat, Front Squat, Hack Squat, Leg Press, Bulgarian Split Squat, Walking Lunge, Leg Extension, Leg Curl, Goblet Squat
Glutes (3)	Hip Thrust, Glute Bridge, Cable Kickback
Calves (3)	Calf Raise, Standing Calf Raise, Seated Calf Raise
Core (5)	Plank, Ab Wheel Rollout, Hanging Leg Raise, Cable Crunch, Crunch
Forearms (2)	Wrist Curl, Farmer's Walk
Cardio (5)	Treadmill Walk/Jog, Bike Sprint, Rowing Machine, Run or Bike, Jump Rope

2.6 Rutinas predefinidas

El **`RoutineController`** incluye cinco rutinas estaticas. No se persisten en base de datos; se devuelven directamente desde codigo:

`ID`	Nombre	Nivel	Objetivo	Dias
`r-beg-strength`	3-Day Beginner Strength	beginner	strength	3
`r-int-hyp`	4-Day Upper/Lower Hypertrophy	intermediate	hypertrophy	4
`r-adv-str`	5-Day Advanced Powerbuilding	advanced	strength	5
`r-beg-gen`	3-Day Beginner General Fitness	beginner	general_fitness	3
`r-int-end`	4-Day Intermediate Endurance + Strength	intermediate	endurance	4

2.7 Configuracion

2.7.1 application.yml (por defecto)

```
server.port: 8000
spring.datasource.url:      jdbc:postgresql://localhost:5432/
    strengthlabs
spring.datasource.username: stlabs
spring.datasource.password: ${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto: validate
compute-engine.url: http://localhost:8001
jwt.secret:         ${JWT_SECRET:dev-secret-key-must-be-at-least-32-
    chars}
jwt.expiration-ms:  86400000          # 24 horas
```

2.7.2 *application-dev.yml*

```
spring.jpa.hibernate.ddl-auto: create-drop   # recrea el esquema en
    cada inicio
logging.level.com.strengthlabs: DEBUG
```

2.7.3 *Variables de entorno requeridas*

Variable	Descripcion	Ejemplo
DB_PASSWORD	Contrasena de PostgreSQL	changeme
JWT_SECRET	Clave HMAC (min 32 chars)	mi-secreto-de-produccion-32ch

3 Motor de computo: Python/FastAPI (puerto 8001)

El motor de computo es un microservicio independiente que expone tres endpoints de calculo. El backend Java lo llama internamente; la app movil **no** lo llama directamente.

3.1 Endpoints

Metodo	Ruta	Descripcion
GET	/health	Comprobacion de salud: {"status":.ºk"}
POST	/compute/fatigue	Recibe historial de sesiones, devuelve metricas ATL/CTL.
POST	/compute/risk	Recibe metricas de fatiga, devuelve evaluacion de riesgo.
POST	/compute/plan	Recibe TSB, devuelve recomendacion de fase y microciclo.

3.2 Modelo de fatiga

3.2.1 TRIMP y carga de sesion

Cuando se dispone de datos de frecuencia cardiaca se usa el modelo TRIMP de Banister:

$$\text{TRIMP} = t \cdot r \cdot e^{\beta r}$$

donde t es la duracion en minutos, $r = (FC_{med} - FC_{rep})/(FC_{max} - FC_{rep})$ es el ratio de frecuencia cardiaca normalizado (acotado a $[0, 1]$) y $\beta = 1{,}92$ para hombres o $\beta = 1{,}67$ para mujeres.

Cuando no hay datos de FC se usa la carga por RPE:

$$\text{Carga}_{\text{RPE}} = t \cdot \text{RPE}$$

3.2.2 EWMA: ATL y CTL

La carga diaria se suaviza con medias moviles exponencialmente ponderadas:

$$S_t = S_{t-1} + \lambda \cdot (\text{carga}_t - S_{t-1})$$

- **ATL** (carga aguda, 7 dias): $\lambda_{ATL} = 2/(7+1) \approx 0{,}25$
- **CTL** (carga cronica, 42 dias): $\lambda_{CTL} = 2/(42+1) \approx 0{,}046$

Metricas derivadas:

$$\text{TSB} = \text{CTL} - \text{ATL}, \qquad \text{ACWR} = \frac{\text{ATL}}{\text{CTL}}$$

3.2.3 Monotonia y strain

Para los ultimos 7 dias:

$$\text{Monotonia} = \frac{\bar{L}}{\sigma_L}, \qquad \text{Strain} = \left(\sum_{i=1}^{7} L_i\right) \cdot \text{Monotonia}$$

donde $\bar{L}$ es la carga media diaria y σ_L su desviacion estandar.

3.2.4 Readiness score

Puntuacion compuesta ponderada en la escala 0–100:

Componente	Peso
ACWR (zona optima 0,8–1,3)	40 %
TSB (positivo = fresco)	35 %
Monotonia (baja = mejor)	15 %
Ramp rate (moderado = mejor)	10 %

3.2.5 Banderas de riesgo de fatiga

Bandera	Condicion
HIGH ACWR	ACWR $\geq$ 1,5
CAUTION ACWR	1,3 $\leq$ ACWR $<$ 1,5
UNDERTRAINING	ACWR $<$ 0,8
HIGH MONOTONY	Monotonia $\geq$ 2,0
HIGH RAMP RATE	Ramp rate $\geq$ 8,0
DEEP FATIGUE	TSB $<$ -30

3.3 Modelo de riesgo

3.3.1 Puntuacion de riesgo de lesion

Funcion de interpolacion lineal por tramos con dos componentes:

- **ACWR** (peso 65 %): lineal desde 0 (ACWR $\leq$ 0,8) hasta 1 (ACWR $\geq$ 2,0), con inflexion en 1,0, 1,3 y 1,5.
- **Ramp rate** (peso 35 %): lineal desde 0 (ramp $\leq$ 5,0) hasta 1 (ramp $\geq$ 12,0).

$$\text{injury_risk} \in [0{,}0,\ 1{,}0]$$

3.3.2 Puntuacion de riesgo de sobreentrenamiento

- **TSB** (peso 70 %): lineal desde 0 (TSB $\geq$ 10) hasta 1 (TSB ≤ -35).
- **Monotonia** (peso 30 %): lineal desde 0 ($\leq$ 1,0) hasta 1 ($\geq$ 3,0).

3.3.3 Puntuacion compuesta

$$\text{composite} = \text{máx}(\text{inj}, \text{over}) \times 0{,}75 + \text{mín}(\text{inj}, \text{over}) \times 0{,}25$$

Niveles de riesgo resultantes:

Nivel	Umbral composite
low	$< 0{,}20$
moderate	0,20–0,44
high	0,45–0,69
critical	$\geq 0{,}70$

3.4 Modelo de periodizacion

3.4.1 Fases de entrenamiento

El modelo implementa periodizacion por bloques (Issurin 2008):

Fase	Microciclo	Criterio de seleccion
RECOVERY	deload	composite $\geq 0{,}70$ o TSB < -35
REALIZATION	peak/taper	dias al evento ≤ 14
TRANSMUTATION	intensification	dias al evento 15–42
ACCUMULATION	base building	CTL < 30 o ACWR $< 0{,}8$
ACCUMULATION	progression	situacion general optima

3.4.2 Multiplicadores de carga

Microciclo	Descripcion	Factor
SHOCK	Semana de choque	$\times 1{,}30$
LOADING	Carga normal	$\times 1{,}05$
MAINTENANCE	Mantenimiento	$\times 0{,}85$
RECOVERY	Descarga (deload)	$\times 0{,}50$
TAPER	Afinamiento	$\times 0{,}60$
COMPETITION	Competicion	$\times 0{,}45$

3.4.3 Salida del plan semanal

```
{
  "phase":                    "accumulation",
  "microcycle_type":          "base_building",
  "week_objective":           "Build aerobic base...",
  "target_weekly_load":       float,
  "target_acwr":              float,
  "projected_tsb_delta":      float,
  "load_adjustment_pct":      float,
  "sessions": [
    {
      "day": 1, "day_name": "Mon",
      "session_type": "Strength Training A",
      "intensity_zone": "z3_tempo",
      "duration_minutes": 60,
```

```
      "rpe_target": 7.0,
      "description": "Full-body compound movements, 3x8-12",
      "is_rest": false,
      "key_session": true
    },
    ...
  ],
  "coach_notes":                [string],
  "periodization_rationale":   string
}
```

4 Frontend: strengthlabs_beta (Flutter)

4.1 Estructura de directorios

```
lib/
  main.dart                     # Punto de entrada (runApp)
  app.dart                      # _AppState: inicializa todas las
     dependencias
  core/
    constants/
      api_constants.dart        # URLs, headers
      app_colors.dart           # Paleta de colores
      app_strings.dart          # Literales de interfaz
    network/
      dio_client.dart           # Cliente Dio + interceptor JWT
    router/
      app_router.dart           # GoRouter con guards de
         autenticacion
    storage/
      token_storage.dart        # flutter_secure_storage (tokens)
      workout_local_storage.dart  # SharedPreferences (calculo
         offline)
  features/
    auth/
      data/auth_repository.dart
      domain/entities/user.dart
      presentation/
        cubit/{auth_cubit, auth_state}.dart
        pages/{login_page, register_page}.dart
    workouts/
      data/
        workout_repository.dart  # CRUD workouts + exercises via API
        mock_workouts.dart       # Datos iniciales (ya no usados en
           repo)
      domain/entities/
        exercise.dart     # Exercise + MuscleGroup enum
        workout.dart      # Workout + WorkoutExercise
        workout_set.dart # WorkoutSet
      presentation/
        cubit/{workouts_cubit, workouts_state}.dart
        cubit/{active_workout_cubit, active_workout_state}.dart
        pages/{workout_list_page, workout_detail_page}.dart
        pages/active_workout_page.dart
```

```
    fatigue/
      data/
        fatigue_repository.dart  # GET /fatigue/summary
        compute_repository.dart  # Calculo local offline
      domain/entities/
        fatigue_summary.dart     # 20+ campos
        fatigue_metrics.dart     # ComputeMetrics tipado
      presentation/
        cubit/{fatigue_cubit, fatigue_state}.dart
        pages/fatigue_dashboard_page.dart
    routines/
      data/routine_repository.dart  # GET /routines
      domain/entities/routine.dart
      presentation/
        cubit/{routines_cubit, routines_state}.dart
        pages/{routines_page, routine_detail_page}.dart
    plan/
      data/plan_repository.dart     # Generacion local de plan
      domain/entities/training_plan.dart
      presentation/
        cubit/{plan_cubit, plan_state}.dart
        pages/plan_page.dart
    export/
      data/export_service.dart      # CSV + XLSX en dispositivo
      presentation/pages/export_page.dart
  shared/
    widgets/
      app_button.dart
      app_text_field.dart
      loading_widget.dart
      main_shell.dart    # Shell de navegacion (BottomNavigationBar)
    utils/
      formatters.dart
      validators.dart
```

4.2 Infraestructura core

4.2.1 DioClient

Clase **`DioClient`** configura una instancia `Dio` con:

- **Base URL:** `ApiConstants.baseUrl` (`http://localhost:8000`).
- **Timeouts:** 15 s de conexion, 15 s de recepcion.
- **Headers por defecto:** `Content-Type: application/json`.
- **Interceptor __AuthInterceptor:**
  1. *onRequest*: lee el access token de **`TokenStorage`** y anade `Authorization: Bearer <token>` a cada solicitud.
  2. *onError* (401): obtiene el refresh token, llama a `POST /auth/refresh` usando una instancia Dio separada (para evitar recursividad), guarda los nuevos tokens y reintenta la solicitud original.

3. Si el refresh falla: limpia los tokens (el router redirige al login).

4.2.2 TokenStorage

Envuelve `flutter_secure_storage` con las siguientes operaciones:

Metodo	Descripcion
`saveTokens(access, refresh)`	Guarda ambos tokens
`getAccessToken()`	Devuelve el access token o null
`getRefreshToken()`	Devuelve el refresh token o null
`clearTokens()`	Elimina ambos tokens
`hasTokens()`	true si el access token no es nulo ni vacio

En Android usa `AndroidOptions(encryptedSharedPreferences: true)`.

4.2.3 ApiConstants

Constante	Valor
`baseUrl`	http://localhost:8000
`register`	/auth/register
`login`	/auth/login
`refresh`	/auth/refresh
`me`	/auth/me
`workouts`	/workouts
`exercises`	/exercises
`fatigueSummary`	/fatigue/summary
`fatigueWeekly`	/fatigue/weekly
`routines`	/routines
`exportXlsx`	/export/xlsx
`exportCsv`	/export/csv
`computeBaseUrl`	http://localhost:8001
`computeFatigue`	/compute/fatigue
`computeRisk`	/compute/risk
`computePlan`	/compute/plan

4.2.4 AppRouter

GoRouter con las siguientes rutas:

Ruta	Descripcion
`/login`	LoginPage (publica)
`/register`	RegisterPage (publica)
`/active-workout`	ActiveWorkoutPage (flotante, no en tabs)
`/workouts`	WorkoutListPage (tab 1)

Ruta	Descripcion
`/workouts/detail/:id`	WorkoutDetailPage
`/routines`	RoutinesPage (tab 2)
`/routines/detail/:id`	RoutineDetailPage
`/fatigue`	FatigueDashboardPage (tab 3)
`/plan`	PlanPage (tab 4)
`/export`	ExportPage (tab 5)

Guard de autenticacion: el router escucha el stream de **`AuthCubit`** a traves de un `ChangeNotifier` y aplica el siguiente redireccionamiento en cada cambio de estado:

- Si `AuthUnauthenticated` o `AuthError` y no es ruta de auth → redirige a `/login`.
- Si `AuthAuthenticated` y esta en `/login` o `/register` → redirige a `/workouts`.
- `AuthInitial` / `AuthLoading` → no redirige (espera a que se resuelva `checkAuthStatus()`).

4.3 Entidades de dominio

4.3.1 User

Campo	Tipo
`id`	String
`name`	String
`email`	String

4.3.2 Exercise

Campo	Tipo	Notas
`id`	String	UUID del backend
`name`	String	
`muscleGroup`	MuscleGroup	enum: chest, back, legs, shoulders, arms, core
`isCustom`	bool	false = global

Los grupos musculares del backend (`biceps`, `triceps`, `forearms`) se mapean a `MuscleGroup.arms`; `glutes`, `calves` se mapean a `MuscleGroup.legs`; `cardio` y `other` a `MuscleGroup.core`.

4.3.3 WorkoutSet

Campo	Tipo	Notas
`id`	String	
`weight`	double	kilogramos
`reps`	int	
`rpe`	double?	escala 6,0–10,0, nullable
`isCompleted`	bool	

Propiedad calculada: `volume = weight * reps`.

4.3.4 WorkoutExercise

Agrupa un **`Exercise`** con su lista de **`WorkoutSet`**. Propiedad calculada: `totalVolume`.

4.3.5 Workout

Campo	Tipo	Notas
`id`	String	UUID del backend
`name`	String	
`date`	DateTime	UTC
`duration`	Duration	de `duration_seconds`
`exercises`	List<WorkoutExercise>	
`notes`	String?	

Propiedades calculadas: `totalVolume`, `totalSets`.

4.3.6 FatigueSummary

Entidad que combina los datos de la API con las metricas del motor de computo:

Campo	Tipo	Descripcion
`overallIndex`	double	Readiness 0–100
`isOvertraining`	bool	Nivel critico de riesgo
`weeklyVolume`	Map<MuscleGroup, double>	Porcentaje 0–100 por grupo
`trend`	List<FatigueDataPoint>	Ultimos 7 dias
`atl`	double	Carga aguda (EWMA 7d)
`ctl`	double	Carga cronica (EWMA 42d)
`tsb`	double	CTL - ATL
`acwr`	double	ATL / CTL
`monotony`	double	Media / desv.est. semanal
`strain`	double	Carga total x monotonia
`rampRate`	double	Delta CTL 7 dias
`readinessScore`	double	0–100 compuesto
`riskFlags`	List<String>	Banderas activas

Campo	Tipo	Descripcion
`injuryRiskScore`	double	0,0–1,0
`overtrainingRiskScore`	double	0,0–1,0
`compositeRiskScore`	double	0,0–1,0
`riskLevel`	String	low / moderate / high / critical
`dominantFactor`	String	Factor determinante del riesgo
`recommendations`	List<String>	Consejos contextuales

Propiedad calculada: `hasComputeData` devuelve `true` si `atl >0` o `ctl >0`.

4.3.7 Routine

Campo	Tipo
`id`	String
`name`	String
`level`	RoutineLevel (beginner / intermediate / advanced)
`goal`	RoutineGoal (strength / hypertrophy / endurance / generalFitness)
`daysPerWeek`	int
`description`	String
`days`	List<RoutineDay>

Cada **`RoutineDay`** contiene una lista de **`RoutineExercise`** (`exercise`, `sets: int`, `repsScheme: String`, `notes: String?`).

4.3.8 TrainingPlan

Generado localmente a partir de las metricas del motor de computo:

Campo	Tipo
`phase`	String
`microcycleType`	String
`weekObjective`	String
`targetWeeklyLoad`	double
`targetAcwr`	double
`projectedTsbDelta`	double
`loadAdjustmentPct`	double
`sessions`	List<PlanSession>
`coachNotes`	List<String>
`periodizationRationale`	String

Cada **`PlanSession`**: `day, dayName, sessionType, intensityZone, durationMinutes, rpeTarget, description, isRest, keySession`.

4.4 Gestion de estado (Cubits)

4.4.1 AuthCubit

Estado	Descripcion
`AuthInitial`	Estado inicial
`AuthLoading`	Operacion en curso
`AuthAuthenticated(user)`	Sesion activa
`AuthUnauthenticated`	Sin sesion
`AuthError(message)`	Error de autenticacion

Metodos:

- `checkAuthStatus()` — comprueba `TokenStorage.hasTokens()`; si hay token llama a `GET /auth/me` y emite `AuthAuthenticated` o `AuthUnauthenticated`.
- `login(email, password)` — llama a `POST /auth/login`, guarda tokens, emite `AuthAuthenticated`.
- `register(name, email, password)` — llama a `POST /auth/register`, guarda tokens, emite `AuthAuthenticated`.
- `logout()` — limpia tokens, emite `AuthUnauthenticated`.

4.4.2 WorkoutsCubit

Estado	Descripcion
`WorkoutsInitial`	Estado inicial
`WorkoutsLoading`	Cargando desde API
`WorkoutsLoaded(list)`	Lista de sesiones disponible
`WorkoutsError(msg)`	Error de red o servidor

Ademas expone el getter `exercises: List<Exercise>` para el selector de ejercicios en la sesion activa.

Metodos:

- `loadWorkouts()` — carga en paralelo `GET /workouts` y `GET /exercises`; ordena por fecha descendente.
- `saveWorkout(workout)` — `POST /workouts`; inserta en cabeza.
- `deleteWorkout(id)` — `DELETE /workouts/{id}`; retira de lista.
- `inRange(range)` — filtra la lista cargada por rango de fechas.
- `findById(id)` — busca en la lista cargada.

4.4.3 ActiveWorkoutCubit

Gestiona la sesion de entreno en curso (sin backend, estado efimero):

- `setName(name)`, `addExercise(exercise)`, `removeExercise(id)`.
- `addSet(exerciseId)`, `removeSet(exerciseId, setId)`.
- `updateSet(...)` — actualiza peso, reps, RPE, completado.
- `finish()` — convierte el estado a entidad **`Workout`** lista para guardarse; descarta series incompletas.

4.4.4 FatigueCubit

Estado	Descripcion
`FatigueInitial`	
`FatigueLoading`	
`FatigueLoaded(summary)`	**`FatigueSummary`** completo
`FatigueError(msg)`	

Metodo `loadSummary()` llama a `GET /fatigue/summary`.

4.4.5 RoutinesCubit

Estado	Descripcion
`RoutinesInitial`	
`RoutinesLoading`	
`RoutinesLoaded(list)`	
`RoutinesError(msg)`	

Metodo `loadRoutines({level})` llama a `GET /routines?level=...` El cubit mantiene una cache interna para `findById(id)`.

4.4.6 PlanCubit

Estado	Descripcion
`PlanInitial`	
`PlanLoading`	
`PlanLoaded(plan)`	**`TrainingPlan`** generado
`PlanError(msg)`	

Metodo `generatePlan(metrics, daysToEvent)` delega en **`PlanRepository`** que usa **`ComputeRepository`** localmente (sin llamada al motor de computo).

4.5 Capa de datos (Repositorios)

4.5.1 AuthRepository

Metodo	Descripcion
`login(email, password)`	POST /auth/login, guarda tokens, llama getCurrentUser()
`register(name, email, password)`	POST /auth/register, guarda tokens, llama getCurrentUser()
`getCurrentUser()`	GET /auth/me, devuelve User
`hasStoredTokens()`	Delega en TokenStorage.hasTokens()
`logout()`	Limpia tokens

Los errores DioException se traducen a `Exception` con mensajes legibles.

4.5.2 WorkoutRepository

Metodo	Descripcion
`getWorkouts()`	GET /workouts, parsea lista de Workout
`createWorkout(w)`	POST /workouts con cuerpo correcto (exercise_id UUID, sets ordenados)
`deleteWorkout(id)`	DELETE /workouts/{id}
`getExercises()`	GET /exercises, parsea lista de Exercise

Incluye helpers privados `_workoutFromApi()`, `_exerciseFromApi()` y `_parseMuscleGroup()` para convertir la nomenclatura del backend al enum local.

4.5.3 FatigueRepository

Metodo `getSummary()` llama a `GET /fatigue/summary` y parsea el cuerpo a **`FatigueSummary`**. Los campos `weekly_volume` (claves de tipo `String`) se agrupan en los 6 grupos del enum `MuscleGroup`.

4.5.4 RoutineRepository

Metodo	Descripcion
`getRoutines({level})`	GET /routines?level=...
`getRoutine(id)`	GET /routines/{id}

Parsea el campo `goal` del JSON (`"general_fitness"`) al valor de enum `RoutineGoal.generalFitness`.

4.5.5 ComputeRepository (calculo offline)

Implementa el mismo modelo EWMA que el motor Python pero en Dart para funcionar sin conexion. Sus resultados alimentan **`PlanRepository`**:

- Construye mapa `dailyLoad = duration_min * avg_rpe` por dia.
- Calcula ATL ($\lambda \approx 0{,}25$) y CTL ($\lambda \approx 0{,}046$) sobre una ventana de 42 dias.
- Deriva TSB, ACWR, monotonia, strain, ramp rate y readiness score.
- Evalua riesgo con umbrales fijos y genera banderas y recomendaciones.

4.5.6 ExportService

`Metodo`	Descripcion
`exportToCsv(workouts)`	Genera archivo CSV en directorio de documentos del dispositivo; comparte via share sheet
`exportToExcel(workouts)`	Genera archivo .xlsx con paquete `excel`; comparte via share sheet

Columnas: Date, Workout, Exercise, Muscle Group, Set #, Weight (kg), Reps, RPE, Volume (kg). Cada fila representa una serie individual.

4.6 Pantallas

Pantalla	Ruta	Descripcion
LoginPage	/login	Formulario email + contrasena; enlace a registro.
RegisterPage	/register	Formulario nombre + email + contrasena.
WorkoutListPage	/workouts	Lista de sesiones ordenadas por fecha. Boton de inicio de nueva sesion.
WorkoutDetailPage	/workouts/detail/:id	Detalle con todos los ejercicios, series, RPE y volumen. Acciones: editar nombre/notas, eliminar.
ActiveWorkoutPage	/active-workout	Cronometro en tiempo real. Selector de ejercicios (por grupo muscular o busqueda). Entrada de peso, reps y RPE por serie. Finalizar o descartar.
RoutinesPage	/routines	Fichas de rutinas filtradas por nivel y objetivo.
RoutineDetailPage	/routines/detail/:id	Tabla de dias y ejercicios de la rutina seleccionada.

Pantalla	Ruta	Descripcion
FatigueDashboardPage	/fatigue	Indice de readiness (0–100), indicador de sobreentrenamiento, grafico de tendencia 7 dias, barras de volumen semanal por grupo muscular, metricas avanzadas (ATL/CTL/TSB/ACWR) y tabla de riesgos.
PlanPage	/plan	Plan semanal generado a partir del estado de fatiga. Tarjetas por dia con tipo de sesion, duracion, zona de intensidad y notas del entrenador.
ExportPage	/export	Selector de rango de fechas. Botones de exportacion CSV y XLSX. Muestra el numero de sesiones en el rango.

5 Flujo de autenticacion

1. **Arranque:** `_AppState.initState()` inicializa **`TokenStorage`**, **`DioClient`** y todos los repositorios. Llama a `AuthCubit.checkAuthStatus()`.
2. **checkAuthStatus:** Si no hay token → emite `AuthUnauthenticated` → router redirige a `/login`. Si hay token → llama a `GET /auth/me`. Si responde 200 → `AuthAuthenticated` → router redirige a `/workouts`. Si falla → `AuthUnauthenticated`.
3. **Login:** `POST /auth/login` → guarda `access_token` y `refresh_token` en **`TokenStorage`** → llama `GET /auth/me` → emite `AuthAuthenticated`.
4. **Peticion protegida:** **`_AuthInterceptor`** lee el access token y anade el header `Authorization: Bearer`.
5. **Token caducado (401):** El interceptor llama a `POST /auth/refresh` en una instancia Dio sin interceptores. Guarda los nuevos tokens y reintenta la solicitud original de forma transparente.
6. **Refresh fallido:** Se limpian los tokens; el router detecta `AuthUnauthenticated` y redirige al login.
7. **Logout:** `AuthCubit.logout()` limpia tokens y emite `AuthUnauthenticated`.

6 Configuracion del entorno de desarrollo

6.1 Requisitos previos

- Docker y Docker Compose
- Java 21 (JDK) y Maven 3.9+
- Python 3.11+

- Flutter SDK 3.x

6.2 Puesta en marcha

6.2.1 1. Infraestructura (PostgreSQL + Redis + Motor de computo)

```
cd StrengthLabsBackend/docker
DB_PASSWORD=changeme docker compose up -d
# Levanta: postgres:5432, redis:6379, compute-engine:8001
```

6.2.2 2. Backend Java

```
cd StrengthLabsBackend
export DB_PASSWORD=changeme
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
# API disponible en http://localhost:8000
# Swagger UI: http://localhost:8000/swagger-ui.html
```

Con perfil `dev` Hibernate recrea el esquema en cada inicio y el **`ExerciseDataSeeder`** inserta el catalogo global.

6.2.3 3. Aplicacion Flutter

```
cd strengthlabs_beta
flutter pub get
flutter run                    # Dispositivo fisico o emulador
# En emulador Android cambiar baseUrl a http://10.0.2.2:8000
```

6.3 Nota sobre URLs en emuladores

En el emulador de Android `localhost` no apunta a la maquina host. Se debe cambiar `ApiConstants.baseUrl` a `http://10.0.2.2:8000`. En simulador iOS y dispositivos fisicos en la misma red se puede usar la IP local del host (p. ej. `http://192.168.1.X:8000`).

7 Decisiones de diseno destacadas

Clean Architecture
: El backend separa claramente dominio, aplicacion, infraestructura y presentacion. Los controladores no contienen logica de negocio; los repositorios de dominio son interfaces.

Motor de computo desacoplado
: Los algoritmos EWMA/ACWR se alojan en un microservicio Python independiente para facilitar su actualizacion sin recompilar el backend Java.

Calculo offline en Flutter
`ComputeRepository` replica los algoritmos del motor en Dart. Esto permite generar el plan semanal sin conexion y sirve de fallback si el motor no esta disponible.

JWT sin estado El backend no mantiene sesiones en memoria ni en BD. Toda la identidad viaja en el token. El refresh token solo se usa para emitir nuevos pares de tokens.

Intercambio de tokens transparente
El interceptor Dio maneja el refresco automaticamente. Las capas superiores (cubits, paginas) jamas ven errores 401.

Rutinas estaticas en el backend
Las cinco rutinas predefinidas estan embebidas en el controlador Java. No requieren migracion de BD y son facilmente extensibles.

Exportacion on-device
El export CSV/XLSX se genera en el dispositivo movil y se comparte via el share sheet nativo. No se transfieren archivos grandes a traves de la red (en cambio, el backend ofrece los mismos endpoints para usos desde web o herramientas externas).

A Catalogo completo de ejercicios globales

Ejercicio	Grupo muscular (backend)
Barbell Bench Press	chest
Incline Barbell Press	chest
Incline Dumbbell Press	chest
Dumbbell Bench Press	chest
Cable Fly	chest
Push-up	chest
Dip	chest
Barbell Deadlift	back
Barbell Row	back
Pendlay Row	back
Dumbbell Row	back
Lat Pulldown	back
Seated Cable Row	back
Pull-up	back
Weighted Pull-up	back
T-Bar Row	back
Romanian Deadlift	back
Overhead Press	shoulders
Dumbbell Shoulder Press	shoulders
Arnold Press	shoulders
Lateral Raise	shoulders

Ejercicio	Grupo muscular (backend)
Front Raise	shoulders
Face Pull	shoulders
Rear Delt Fly	shoulders
Barbell Curl	biceps
Dumbbell Curl	biceps
EZ Bar Curl	biceps
Hammer Curl	biceps
Preacher Curl	biceps
Incline Dumbbell Curl	biceps
Tricep Dip	triceps
Tricep Pushdown	triceps
Skull Crusher	triceps
Close-Grip Bench Press	triceps
Overhead Tricep Extension	triceps
Barbell Back Squat	legs
Front Squat	legs
Hack Squat	legs
Leg Press	legs
Bulgarian Split Squat	legs
Walking Lunge	legs
Leg Extension	legs
Leg Curl	legs
Goblet Squat	legs
Hip Thrust	glutes
Glute Bridge	glutes
Cable Kickback	glutes
Calf Raise	calves
Standing Calf Raise	calves
Seated Calf Raise	calves
Plank	core
Ab Wheel Rollout	core
Hanging Leg Raise	core
Cable Crunch	core
Crunch	core
Wrist Curl	forearms
Farmer's Walk	forearms
Treadmill Walk/Jog	cardio
Bike Sprint	cardio
Rowing Machine	cardio
Run or Bike	cardio
Jump Rope	cardio

B Rutinas predefinidas: detalle de dias

B.1 r-beg-strength: 3-Day Beginner Strength

Dia	Ejercicio	Series	Reps
Day A — Full Body	Barbell Back Squat	3	5
	Barbell Bench Press	3	5
	Barbell Deadlift	1	5
	Overhead Press	3	5
	Barbell Row	3	5
Day B — Full Body	Barbell Back Squat	3	5
	Overhead Press	3	5
	Barbell Deadlift	1	5
	Barbell Bench Press	3	5
	Barbell Row	3	5
Day C — Full Body	Barbell Back Squat	3	5
	Barbell Bench Press	3	5
	Barbell Deadlift	1	5
	Overhead Press	3	5
	Barbell Row	3	5

B.2 r-int-hyp: 4-Day Upper/Lower Hypertrophy

Dia	Ejercicio	Series	Reps
Upper A (Strength)	Barbell Bench Press	4	4–6
	Barbell Row	4	4–6
	Incline Dumbbell Press	3	8–12
	Cable Row	3	8–12
	Lateral Raise	3	12–15
	Tricep Pushdown	3	10–15
	Dumbbell Curl	3	10–15
Lower A (Strength)	Barbell Back Squat	4	4–6
	Romanian Deadlift	3	8–10
	Leg Press	3	10–12
	Leg Curl	3	10–12
	Calf Raise	4	12–15
Upper B (Volume)	Incline Barbell Press	4	8–12
	Lat Pulldown	4	8–12
	Dumbbell Shoulder Press	3	10–12
	Cable Fly	3	12–15
	Face Pull	3	15–20
	EZ Bar Curl	3	10–12
	Skull Crusher	3	10–12
Lower B (Volume)	Barbell Deadlift	4	4–6
	Front Squat	3	6–8
	Walking Lunge	3	10–12
	Leg Extension	3	12–15
	Standing Calf Raise	4	12–15
	Plank	3	60s

Las rutinas **r-adv-str**, **r-beg-gen** y **r-int-end** siguen el mismo formato y estan disponibles en el endpoint `GET /routines` del backend con sus respectivos IDs.